

Appendix A. *Annotations for Concurrency*

We've used annotations such as `@GuardedBy` and `@ThreadSafe` to show how thread-safety promises and synchronization policies can be documented. This appendix documents these annotations; their source code can be downloaded from this book's website. (There are, of course, additional thread-safety promises and implementation details that should be documented but that are not captured by this minimal set of annotations.)

A.1. Class Annotations

We use three class-level annotations to describe a class's *intended* thread-safety promises: `@Immutable`, `@ThreadSafe`, and `@NotThreadSafe`. `@Immutable` means, of course, that the class is immutable, and implies `@ThreadSafe`. `@NotThreadSafe` is optional—if a class is not annotated as thread-safe, it should be presumed not to be thread-safe, but if you want to make it extra clear, use `@NotThreadSafe`.

These annotations are relatively unintrusive and are beneficial to both users and maintainers. Users can see immediately whether a class is thread-safe, and maintainers can see immediately whether thread-safety guarantees must be preserved. Annotations are also useful to a third constituency: tools. Static codeanalysis tools may be able to verify that the code complies with the contract indicated by the annotation, such as verifying that a class annotated with `@Immutable` actually is immutable.

A.2. Field and Method Annotations

The class-level annotations above are part of the public documentation for the class. Other aspects of a class's thread-safety strategy are entirely for maintainers and are not part of its public documentation.

Classes that use locking should document which state variables are guarded with which locks, and which locks are used to guard those variables. A common source of inadvertent non-thread-safety is when a thread-safe class consistently uses locking to guard its state, but is later modified to add either new state variables that are not adequately guarded by locking, or new methods that do not use locking properly to guard the existing state variables. Documenting which variables are guarded by which locks can help prevent both types of omissions.

`@GuardedBy(lock)` documents that a field or method should be accessed only with a specific lock held. The `lock` argument identifies the lock that should be held when accessing the annotated field or method. The possible values for `lock` are:

- `@GuardedBy("this")`, meaning the intrinsic lock on the containing object (the object of which the method or field is a member);
- `@GuardedBy("fieldName")`, meaning the lock associated with the object referenced by the named field, either an intrinsic lock (for fields that do not refer to a `Lock`) or an explicit `Lock` (for fields that refer to a `Lock`);
- `@GuardedBy("ClassName.fieldName")`, like `@GuardedBy("fieldName")`, but referencing a lock object held in a static field of another class;
- `@GuardedBy("methodName()")`, meaning the lock object that is returned by calling the named method;
- `@GuardedBy("ClassName.class")`, meaning the class literal object for the named class.

Using `@GuardedBy` to identify each state variable that needs locking and which lock guards it can assist in maintenance and code reviews, and can help automated analysis tools spot potential thread-safety errors.